Data Driven Processing with CAS in SAS Viya

Hands-On Workshop Instructions

Stephen Foerster
Data Engineering and Decisioning Lead, SAS Education

Copyright © 2025, SAS Institute Inc., Cary, North Carolina, USA. ALL RIGHTS RESERVED.

Explore the SAS Studio Interface

1. Launch Google Chrome from the desktop and select **SAS Studio** from the Bookmarks bar.

2. Enter **student** in the **User ID** field and **Metadata0** in the **Password** field. Click **Sign in**. Select **Yes** when prompted to opt in to all assumable groups.

3. Click the **applications menu** icon in the upper left corner. The applications menu shows all the available applications. Depending on your environment, these applications might differ. Select **Develop Code and Flows** to open SAS Studio. We use SAS Studio to write and submit code in this workshop.

4. The Start Page tab is open by default, and it provides convenient links to start writing a program or use other point-and-click tools, including importing and querying data. Click **Program in SAS** to open a new tab with the Program Editor. Each item that you open in SAS Studio has its own tab.

Section 1: Introduction to CASL Variables

Start a CAS Session

5. Select **SAS Server** on the left and expand **Files** > **Home** > **Courses** > **DDPWC**. Double-click **DDPWC.sas**.

6. Execute the code under the ***** Start a CAS Session ***** commend header.

```
*** Start a CAS Session ***;
cas mysess;
caslib _all_ assign;
```

This code starts your own personal session on the CAS server. The log will show that the CAS session started successfully. The code also assigns SAS libraries to any existing CAS libraries on the CAS server. You can see these in the log or by clicking the **Library Connections** tab on the left.

Run a Simple SAS Macro

7. Execute the code under the ***** SAS Macro: Print Summary or Detail Report ***** commend header.

```
**************************************************;
*** SAS Macro:  Print Summary or Detail Report ***;
**************************************************;

%macro conLogic;
    %let aggreg8=Y;

    %if &aggreg8=Y %then %do;
        Data public.Agg_Profit ;
```

Copyright © 2025, SAS Institute Inc., Cary, North Carolina, USA. ALL RIGHTS RESERVED.

```
            Set public.Orion_Profit ;
                keep Product_Category Customer_Group aggProfit;
                by Product_Category Customer_Group;
                retain aggProfit;
                if first.Customer_Group then AggProfit = 0;
                aggProfit = aggProfit + Profit;
                if last.Customer_Group then output;
            run;

            Proc Print data=public.Agg_Profit;
            run;
        %end;
        %else %do;
            Proc Print data=public.Orion_Profit (obs=100);
            run;
        %end;
    %mend;

    %conLogic;
```

The code creates a SAS macro, **conLogic** which either prints detailed or aggregated data depending on the value of the **aggreg8** macro variable. If **Y**, it prints aggregated data. If **N**, detailed.

Change the value of **aggreg8** and see the effect.

Run the Equivalent Logic Using CASL

8. Execute the code under the ***** CASL: Print Summary or Detail Report ***** commend header.

```
    ************************************************;
    *** CASL:  Print Summary or Detail Report     ***;
    ************************************************;

    Proc CAS;
        aggreg8='N';

        if aggreg8='Y' then do;
            datastep.runCode submit / code="
                Data public.Agg_Profit (replace=yes) ;
                Set public.Orion_Profit ;
                keep Product_Category Customer_Group aggProfit;
                    by Product_Category Customer_Group;
                    retain aggProfit;
                    if first.Customer_Group then AggProfit = 0;
                    aggProfit = aggProfit + Profit;
                    if last.Customer_Group then output;
                run;";

            table.fetch / to=100
                table={caslib="public" name="Agg_Profit"};
        end;
        else do;
            table.fetch / to=100
                table={caslib="public" name="Orion_Profit"};
```

Copyright © 2025, SAS Institute Inc., Cary, North Carolina, USA. ALL RIGHTS RESERVED.

```
        end;
    quit;
```

Like the **conLogic** macro, this CASL code either prints detailed or aggregated data depending on the value of the **aggreg8** variable. However, this time **aggreg8** is a CASL variable. Again, change the value of **aggreg8** and see the effect.

This comparison illustrates that CASL variables and macro variables serve many of the same purposes. Particularly, we can use either to drive programming logic.

Section 2: CASL Variables, More Detail

Run Some CASL to Learn about CASL Variables

9. Execute the code under the ***** Basic Types ***** commend header.

```
    ******************************************************;
    *** Basic Types                                    ***;
    ******************************************************;

    Proc CAS;
        /* CASL varaibles can be strings */
        custName = "Mary Smith";

        print custName " likes SAS";

        /* CASL variables can be numeric  */
        y = 99;

        print "y plus 50 = " y + 50;

        /* Boolean  */
        exists = true;

        if exists then print exists;

        /* String array */
        colNames = {"make", "model", "type"};

        print "The 2nd column is " colNames[2];

        /* Numeric array */
        percentiles = {5, 25, 50, 75, 95};

        percentiles = percentiles + 99;

        print "The report shows: " percentiles[1:6];

        /* Multi-dimensional array */
```

Copyright © 2025, SAS Institute Inc., Cary, North Carolina, USA. ALL RIGHTS RESERVED.

```
        multiDim = { {1, 2}, {3, 4}, {5, 6} };

        print "The 2nd unit of the 3rd pair is " multiDim[3,2];
    quit;
```

The code creates and prints various CASL variables. It shows string types, numeric types, and Boolean types as well as arrays. It also shows some basic CASL variable manipulation as well as referencing techniques.

Run Some CASL to Learn about CASL Dictionary Variables

10. Execute the code under the ***** Basic Dictionary ***** commend header.

```
    ******************************************************;
    *** Basic Dictionary                                ***;
    ******************************************************;

    proc cas;
        /* Create a dictionary variable */
        myTable = {name="cars", caslib="casuser", replace=true};

        /* Display the structure in the log */
        describe myTable;

        /* Access by individual keys */
        print "Table name is " myTable.name;
    quit;
```

The code creates, describes, and prints a basic dictionary CASL variable. The PRINT statement shows how to reference dictionary members.

Run Some CASL to Learn about Nested CASL Variables

11. Execute the code under the ***** Nested Dictionary ***** commend header.

```
    ******************************************************;
    *** Nested Dictionary                               ***;
    ******************************************************;

    proc cas;
        /* Create a dictionary variable */
        myCar = {name="Odyssey", Features = {wheels=4, seats=7, color="red"}};

        /* Display the structure in the log */
        describe myCar;

        /* Access with an individual key */
        print myCar.features.color;
   quit;
```

Copyright © 2025, SAS Institute Inc., Cary, North Carolina, USA. ALL RIGHTS RESERVED.

The code creates, describes, and prints a nested dictionary CASL variable. The variable contains a string variable and another dictionary. The PRINT statement shows how to reference nested dictionary members.

Section 3: CAS Actions, Briefly

Run Some CASL to Learn about CAS Actions

12. Execute the code under the ***** Section 3: CAS Actions, Briefly ***** commend header.

```
************************************************;
*** Section 3:  CAS Actions, Briefly          ***;
************************************************;

    /* Run the FileInfo action */
    proc cas;
      table.fileinfo / caslib="public";
    quit;

    /* Run the Summary action */
    proc cas;
      simple.summary
        table={caslib="Public" name="Orion_Profit"
               groupBy="Product_Category"}
            inputs={"Profit"}
            subset={"sum"};
    quit;
```

The code runs two different CAS actions, **table.fileInfo** and **simple.Summary**. Running the CAS actions one at a time allows users to understand their use. The first CAS action lists the tables in a CASlib. The second action generates summary statistics from a dataset.

Section 4: CASL Result Variables

Run PROC DATASETS to obtain Table Information

13. Execute the code under the ***** Base SAS: Obtain SAS Library Information ***** commend header.

```
    ************************************************;
    *** Base SAS:  Obtain SAS Library Information ***;
    ************************************************;

    *** Write SAS Library Information to "Results" ***;
```

Copyright © 2025, SAS Institute Inc., Cary, North Carolina, USA. ALL RIGHTS RESERVED.

```
proc datasets lib=public;
quit;

*** Write SAS Library Information to a Table ***;
proc datasets lib=public noprint;
    contents data=_all_ out=work.pubData;
quit;
```

The first step runs a PROC DATASETS which displays information about the tables in the PUBLIC library. The second step runs the same procedure but sends its results to an output SAS table.

Run the tableInfo CAS Action to obtain Table Information

14. Execute the code under the ***** CASL: Obtain CAS Library Information ***** commend header.

```
*** Write SAS Library Information to "Results" ***;
proc cas;
    table.tableinfo / caslib="public";
run;

*** Write SAS Library Information to a CASL variable ***;
proc cas;
    table.tableinfo result=tabInf / caslib="public";

    describe tabInf;
    print tabInf.tableInfo;
    print tabInf.TableInfo[2,"Name"];

quit;
```

Like the previous exercise, the first step runs a tableInfo action which displays information about the tables in the PUBLIC library and the second step runs the same action but sends its results to an output CASL variable, **tabInf**.

The tabInf CASL variable is a dictionary with a nested table inside. The DESCRIBE statement shows the structure while the first PRINT statement shows its contents. The second PRINT statement shows how to reference a value within the nested table.

Section 5: Load All Files in a Directory to CAS

Use SAS Macro to load all CSV Files from a Directory

15. Execute the code under the ***** SAS Macro: Load All CSV Files in a Directory ***** commend header.

```
*****************************************************;
*** SAS Macro:  Load All CSV Files in a Directory ***;
```

Copyright © 2025, SAS Institute Inc., Cary, North Carolina, USA. ALL RIGHTS RESERVED.

```
*****************************************************;

%macro import_csv_dir(path);
    %let rc = %sysfunc(filename(mydir, &path));
    %let did = %sysfunc(dopen(&mydir));
    %let memcnt = %sysfunc(dnum(&did));

    %do i = 1 %to &memcnt;
        %let name = %qsysfunc(dread(&did, &i));
        %if %qupcase(%qscan(&name, -1, .)) = CSV %then %do;
            %let dsn_name = %qscan(&name, 1, .);
            proc casutil;
                load file="&path/&name" casout="&dsn_name"
                    outcaslib="casuser" importoptions=(filetype="CSV")
                    replace ;
            quit;
        %end;
    %end;

    %let rc = %sysfunc(dclose(&did));
%mend import_csv_dir;

%import_csv_dir(/home/student/Courses/DDPWC);
```

This code creates and executes a SAS macro that loads all CSV files in a directory to CAS. First, the macro uses %sysfunc functions to create a SAS filename to the directory and count the number of files inside. Then it iterates over the files one at a time and loads them with PROC CASUTIL.

Use CASL to load all CSV Files from a Directory

16. Execute the code under the ***** CASL: Load All CSV Files in a Directory ***** commend header.

```
*****************************************************;
*** CASL:  Load All CSV Files in a Directory      ***;
*****************************************************;

caslib mylib path="/home/student/Courses/DDPWC";
libname mylib cas caslib=mylib;

/* See the FileInfo action */
proc cas;
  table.fileinfo / caslib="mylib";
quit;

/* Loop over the FileInfo results with LoadTable */
proc cas;
  table.fileinfo result=filesToLoad/ caslib="mylib";

  do i = 1 to dim(filesToLoad.FileInfo);
     fname = filesToLoad.FileInfo[i,'Name'];
     if scan(fname,2,'.')='csv' then do;
        table.loadtable /
           caslib="mylib"
           path=fname;
```

Copyright © 2025, SAS Institute Inc., Cary, North Carolina, USA. ALL RIGHTS RESERVED.

```
        end;
      end;
    quit;
```

This code creates a CASLib to the directory and then uses the fileInfo action to push the information about the files in that CASLib to the **filesToLoad** CASL variable. A CASL Do loop then iterates over those files and loads any CSV files using the **loadTable** CAS action.

Section 6: Use a Parameter Table to Drive Processing

Run CASL Code that Drives Processing from a Parameter Table

17. Execute the code under the ***** Section 6: Use a Parameter Table to Drive Processing ***** commend header.

```
/* Load the Parameter Table to CAS */
proc cas;
    table.loadtable / caslib="mylib" path="ParameterTable.sas7bdat";

    table.fetch / table={caslib="mylib" name="ParameterTable"};
quit;


/* Read the Orion_Profit condition and apply it to a report */
proc cas;
    table.fetch result=OrionParm /
        table={caslib="mylib" name="ParameterTable"
               where="process='Orion_Profit'"};

    whereClause=OrionParm.Fetch[1, 'Condition'];

    simple.summary
        table={caslib="Public" name="Orion_Profit"
               groupBy="Product_Category"
               where=whereClause}
        inputs={"Profit"}
        subset={"sum"};
  quit;
```

This code loads the **ParameterTable** SAS table to CAS. Then performs a **Fetch** action to show its contents.

The second code block performs a **Fetch** where the Process field is equal to **Orion_Profit.** The **whereClause** CASL variable is set to the Condition value from that row. This makes **whereClause** equal to 'Product_Line="Sports".'

This WHERE clause is then applied to a **simpleSummary** action demonstrating how code, values, and conditions can be applied from tables making processes data driven.

Copyright © 2025, SAS Institute Inc., Cary, North Carolina, USA. ALL RIGHTS RESERVED.

Finally, please run the **cas mysess terminate** statement to terminate your CAS session.

Copyright © 2025, SAS Institute Inc., Cary, North Carolina, USA. ALL RIGHTS RESERVED.